IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 30 August 2022, accepted 5 September 2022, date of publication 14 September 2022, date of current version 27 September 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3206537

RESEARCH ARTICLE

MARL-Based Cooperative Multi-AGV Control in Warehouse Systems

HO-BIN CHOI[1], JU-BONG KIM[1], YOUN-HEE HAN[1], (Member, IEEE), SE-WON OH[2], AND KWIHOON KIM[3]

[1]Future Convergence Engineering, Korea University of Technology and Education, Cheonan 31253, South Korea
[2]Department of Knowledge-Converged Super Brain Convergence Research, ETRI, Daejeon 34129, South Korea
[3]Department of Artificial Intelligence Convergence Education, Korea National University of Education, Cheongju 28173, South Korea

Corresponding author: Youn-Hee Han (yhhan@koreatech.ac.kr)

This work was supported by the two Basic Science Research Programs through the National Research Foundation of Korea (NRF) funded by the Ministry of Education under Grant NRF-2020R1I1A3065610 and Grant NRF-2018R1A6A1A03025526.

ABSTRACT Automated guided vehicles (AGVs) are essential components for the automation of fulfillment centers, a type of warehouse, where goods are stored on shelves and carried by AGVs. To increase the productivity in inventory management, a well-organized cooperative path control is required to transport goods to the designated picking stations. In this paper, we propose a QMIX-based scheme for the cooperative path control of multiple AGVs. Although QMIX is the one of popular cooperative multi-agent reinforcement learning algorithms, we find that QMIX alone was not enough to increase productivity in warehouse systems. So, we develop two novel techniques that can be used with QMIX: 1) *sequential action masking* that eliminates all the collision cases and 2) *additional local loss* that improves collaboration of individual AGVs. They help to encourage the AGVs to cooperate more for high productivity. By extensive simulations, we present the superiority of the proposed scheme on several layouts in fulfillment centers. The effect of cooperation among AGVs in the proposed scheme is verified through the comparison study with the existing algorithms. Additionally, we show the generalization performance by investigating the reusability of the model trained with the proposed scheme.

INDEX TERMS AGV, fulfillment center, multi-agent reinforcement learning, path planning.

I. INTRODUCTION

Recently, with the growth of the e-commerce market, the competitiveness of fulfillment services has been proven and large-scale fulfillment centers are required. The fulfillment center is a warehouse for storing goods and a system that responds quickly to customer orders. In fulfillment centers, AGVs that enable automation are essential components to increase productivity. When packing goods for delivery to customers, workers can simply instruct AGVs to bring the items to the picking station. Therefore, AGV's transport path control capability affects customer satisfaction largely, and an efficient path control scheme is required for multiple AGVs to move simultaneously.

Typical methods of AGV path planning are divided into global path planning such as genetic algorithm and artificial bee colony algorithm, and partial path planning such as artificial potential field method and A* algorithm [1]. However, both of them have several disadvantages: 1) it is not suitable for real-time operation because of the large amount of computation required; 2) it is easy to converge on a local optimal solution; 3) it is less adaptable to the changed environment; and 4) it is difficult to expand to a large number of AGVs because of the scalability issue. Furthermore, each AGV needs strict and careful consideration of conflicts between them. If greedy path control is applied to all the AGVs, the probability of collisions would increase exponentially, and the system would suffer catastrophic performance degradation. However, the AGVs should not be too concerned with collisions and control their path as short as possible without creating any congestion or bottlenecks.

The associate editor coordinating the review of this manuscript and approving it for publication was Qiang Li.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

Reinforcement learning (RL) is attracting attention as it is being applied as a new approach to various problems [2], [3]. On the other hand, most real-world problems, including ones in multi-AGV warehouses, occur when many entities cooperate or compete with each other rather than a single entity does [4], [5], [6]. In most environments, general single-agent RL, which can control one entity, cannot simply be extended to handle multiple entities because other entities are not recognized. However, multi-agent RL, in which entities recognize each other through centralized training or communication between entities, can be applied to various fields where multiple entities interact [7], [8], [9]. Within the AGV warehouse, numerous AGVs plan their paths to complete their missions. Since AGVs must recognize or anticipate each other's paths, the difficulty of path control increases exponentially as the number of AGVs increases or the warehouse becomes cramped. In this study, we approached this problem to be able to plan paths in real-time by using multi-agent RL to solve it efficiently. In addition, we adopted the centralized training with decentralized execution (CTDE) framework suitable for general decentralized systems [10]. Through the CTDE framework, each AGV plans its own path while trying to collaborate with other AGVs, and they finally increase the productivity of the warehouse.

The QMIX is a popular algorithm in the field of multi-agent RL and has a structure that is suitable for solving cooperative tasks. Each AGV can be set as an individual agent, and its path, considering collaboration, can be controlled in real-time using the QMIX. However, the individual sacrifices of each agent can sometimes be ignored, because the QMIX aims to maximize the rewards of the entire team. Therefore, we propose a scheme that improves upon the QMIX by applying additional techniques to enhance feedback for individual agents.

For conducting the simulations, we created several fulfillment center layouts and analyzed the results based on three metrics that are considered important in fulfillment centers. The design feasibility of the multi-agent RL environment modeling the AGV warehouse can be verified through these simulations. The simulations were divided into two main categories. First, analysis of coordination performance in given fulfillment center layouts. To prove the superiority of the coordination performance of the proposed scheme, it was compared with the existing QMIX and independent Q-learning (IQL). We further emphasized the impact of the proposed scheme by changing only the number of AGVs in the same fulfillment center layout in the simulation. AGVs have a huge disadvantage in that they can collide with one another. Therefore, they need to carefully determine their travel paths because some AGVs may be bottlenecked or jammed in a narrow area. For this analysis, we present heat maps that visualized the movements of the trained AGVs for a more visible analysis. Second, analysis of generalization performance for unseen fulfillment center layouts. The robustness and adaptability of the trained model can be investigated through this simulation. Even if the layout of fulfillment center changed, the goal of the AGVs remained the same, implying they would be able to adapt accordingly to some extent. If the generalization performance could be guaranteed, the trained model could be reused to train a new fulfillment center layout of complex structures.

The main contributions of this study are summarized as:

1) we formalize the AGV warehouses as a decentralized partially observable Markov decision process (Dec-POMDP) to solve the task with cooperative multi-agent RL.
2) we remove all the possible collision cases by introducing sequential action masking.
3) we ensure that all the AGVs behave systematically through specific individual feedback.

The rest of the paper is organized as follows. In Section II, related studies that attempted to apply RL to AGVs are discussed, and the background of the QMIX is summarized. The system model of the designed AGV warehouse is described in Section III, and the proposed schemes are presented in Section IV. Section V interprets the simulation results and Section VI discusses the proposed schemes. The conclusion is drawn in Section VII.

II. RELATED WORK

A. AUTOMATED GUIDED VEHICLES IN WAREHOUSES

Amazon Robotics, formerly Kiva Systems, deals with resource allocation problems, including decision-making under uncertainty, robot path planning, and scheduling in AGV warehouses. Fortunately, they provide a natural multi-agent AGV warehouse scenario for coordinated autonomy and decentralized decision-making [11]. These insights have encouraged research across various fields. Among them, we focused on real-time path control using multi-agent RL. The multi-AGV path planning problem can be interpreted in various ways, and many algorithms have been proposed to tackle it [12], [13], [14]. Recently, studies to control the multi-AGV path in real-time by applying single-agent RL have been attempted, achieving good results that outperformed those of traditional algorithms [15], [16]. To the best of our knowledge, there are no studies on the multi-AGV path control using multi-agent RL [17]. We adopted the CTDE framework to ensure decentralized decision-making and realize coordinated autonomy through communication between agents using multi-agent RL rather than single-agent RL.

B. MULTI-AGENT DEEP REINFORCEMENT LEARNING

In single-agent RL, a representative value-based algorithm with high sample efficiency is the DQN based on the Q-learning [18], [19]. The Q-learning can be simply extended to the IQL to be applied in multi-agent RL [20]. However, the IQL cannot prevent non-stationarity from worsening because of the absence of communication between agents. The value deposition networks (VDN) calculate the joint action value of the entire team by simply summing the action values of all the agents [21]. This additivity constraint was relaxed

by the QMIX as a monotonicity constraint [22]. The QMIX estimated the entire team's joint action value in a sophisticated nonlinear method using hypernetworks [23] rather than in a linear method such as VDN; hypernetworks can derive a richer representational capacity of the joint action value function (i.e., the mixing network) using extra state information. The monotonicity constraints of the mixing network that extracts the joint action value were ensured by constraining the weights created by the hypernetworks to be non-negative. Since this factorization from the entire team's joint action value to each agent's action value facilitates the distributed execution of multi-agent, it is suitable for real-world problems in which multiple entities interact, such as multiple AGVs in warehouse systems.

III. SYSTEM MODEL

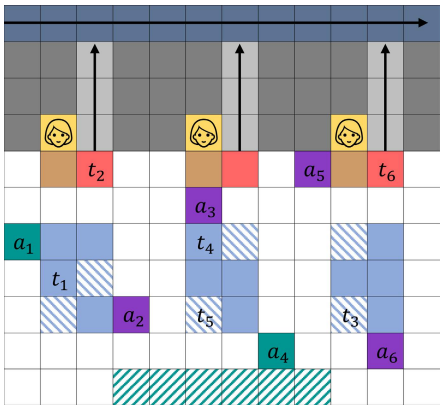
We consider a multi-agent environment to solve a task that requires collaboration between agents. This task can be formulated as a Dec-POMDP [24], represented by a tuple $\langle A, S, O, Z, U, P, R_g, R_l, \gamma \rangle$. At each time step t, the environment outputs a global state $s_t \in S$, and observations $o_t^i = O(s_t, i) \in Z$, where S denotes the global state space, Z denotes the observation space for each agent $i \in \{1, \ldots, n\} \equiv A$, and $O : S \times A \rightarrow Z$ denotes the observation function respectively. Each agent uses its own observations to select an action $u_t^i \in U$, where U denotes the action space for each agent $i \in \{1, \ldots, n\}$. The environment executes the joint action $\boldsymbol{u}_t \in \boldsymbol{U} \equiv U^n$ for all the agents, resulting in a transition according to the state transition function $P(s_{t+1} \mid s_t, \boldsymbol{u}_t) : S \times \boldsymbol{U} \times S \rightarrow [0, 1]$. This transition is completed with a global reward $r_t = R_g(s_t, \boldsymbol{u}_t)$ and individual rewards $r_t^i = R_l(s_t, u_t^i)$, where $R_g : S \times \boldsymbol{U} \rightarrow \mathbb{R}$ denotes the global reward function and $R_l : S \times U \rightarrow \mathbb{R}$ denotes the individual reward function respectively.

In this process, each agent tries to increase its own individual reward, which is aggregated into the global reward, ultimately aiming to maximize the discounted cumulative global reward $\sum_{k=0}^{\infty} \gamma^k r_{t+k}$. Each agent generates its own observation-action history $\tau^i \in T \equiv (Z \times U)^*$ subject to a stochastic policy $\pi^i(u^i \mid \tau^i) : T \times U \rightarrow [0, 1]$, where $(Z \times U)^*$ represents the set of all possible observation-action histories. The joint policy π consisting of each agent's policy π^i has a joint action-value function, which is formulated as:

$$Q^{\pi}(s_t, \boldsymbol{u}_t) = \mathbb{E}_{s_{t+1:\infty}, \boldsymbol{u}_{t+1:\infty}} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} | s_t, \boldsymbol{u}_t \right]. \quad (1)$$

A. PROBLEM STATEMENT

Fig. 1 visually shows the configuration of a typical fulfillment center to be used in this study. Note that this configuration is grid-based and the components that can be located in each grid cell are shown in Fig. 1(b). The overall process of the system is summarized as follows: 1) when the system is activated, all the workers instruct the AGVs to bring the shelves where the items needed to pack the product are stored; 2) the assigned AGVs move to the requested shelf and lift it up; 3) the AGVs carry the shelf to the picking station entrance



(a) Snapshot of an example of the fulfillment center

	Floor		Conveyor (forwarding to next station)
	Shelf Storage		No-Access Zone
	Shelf		Worker
	Charging Station		Picking Station Queue (PSQ)
	AGV		Picking Station Entrance (PSE)
	AGV + Shelf		Picking Station eXit (PSX)

(b) Component representation

FIGURE 1. Configuration of a typical fulfillment center. The AGVs wait at the charging station before the system is activated. When the system is activated, the AGVs transport the shelves along the floors to the picking stations or shelf storages at the request of workers. The workers pick items from shelves carried by AGVs and pack the products for delivery to customers. The packed products are forwarded along the conveyor to the next station. In (a), there are 6 AGVs, and the destination of each AGV a_i is indicated by t_i.

(PSE) of the picking station; 4) each worker picks items from the shelves carried by the AGVs and packs the products, and 5) after the work at the picking station is completed, the AGVs transport the shelf to an available shelf storage and lay it down. Until the system is deactivated, the instructions of the workers are continuous, and the AGVs repeat steps 2) to 5). When the system is deactivated, all the AGVs move to the charging station. Only 2), 3), and 5) are tasks that require path control of AGVs.

In this study, we do not cover the entire process and focus only on the path control of the AGVs. In the entire process, the tasks related to the path control of the AGVs form a cycle consisting of AGV states [Moving to Shelf, Moving to PSE, Moving to Storage]. Each AGV can be in a different state, but an AGV cannot be in multiple states at the same time. We assume that, for simplicity, tasks other than AGV path control are as follows: workers at all picking stations constantly request to bring shelves to the AGVs; each request is assigned to a random available AGV; each AGV returns the shelf which the picking task is completed to a random available shelf storage; and all the AGVs have enough batteries to operate until the system is deactivated. Note that AGVs, shelves, and shelf storages are not randomly selected in actual AGV warehouses; therefore, the environment presented in

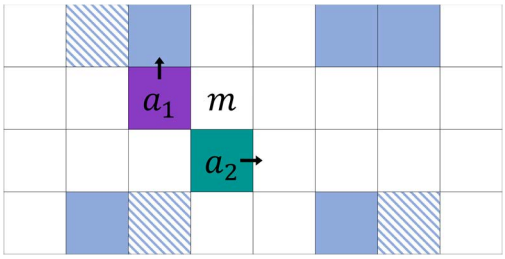


FIGURE 2. Example of sequential action masking. The looking direction of each agent is determined by its own previous action. Assume that agent a_1's looking direction is north and agent a_2's looking direction is east. The available actions of agent a_1 are [stop, move right, move backward, move left], where action move forward is masked as agent a_1 is holding a shelf and cannot get under another shelf. If agent a_1 chooses action move right, the location m becomes a no-access zone for other agents. Then, the available actions of agent a_2 are [stop, move forward, move right, move backward], where action move left is masked by agent a_1.

this paper is a more general one because they are randomly selected.

B. STATE AND OBSERVATION REPRESENTATION

We treat each AGV as an independent agent to comply with decentralized decision-making. At each time step, each agent receives its own observation that is generated from the environment. The observation of each agent is composed of two-channel information on the surrounding 9×9 area centered on itself, forming a shape of $2 \times 9 \times 9$. The first channel consists of values of 1.0 if the agent can move to the corresponding location; otherwise, 0.0. The second channel consists of values normalized to the range of 0.0 to 1.0 for the remaining Manhattan distances from each location to the agent's target location. At the same time as the observations, the environment also creates a state on the information about the entire area of the fulfillment center layout. The only first channel consists of each location's usage normalized to the range of 0.0 to 1.0 for all the agents. The size of the state is determined by the grid size of the fulfillment center layout. Since this state is later used to update the model of the agents, we decided that it was reasonable to provide observations about the surrounding areas of 9×9 to each agent for their own decision-making. If the size of the observations is too small or too large, the ability of the agents to perceive the surrounding situation is impaired.

C. ACTION REPRESENTATION

Each agent selects an action that can move one grid cell from its location using its own observation provided by the environment. The selected action is performed based on the looking direction of the agent, and the action space of all the agents is defined as follows: {stop, move forward, move right, move left, move backward}. At the beginning of the episode, each agent's initial looking direction is arbitrarily set; thereafter, it is determined by the action it will choose. In this mechanism, the observation is rotated with respect to the looking direction of the agent.

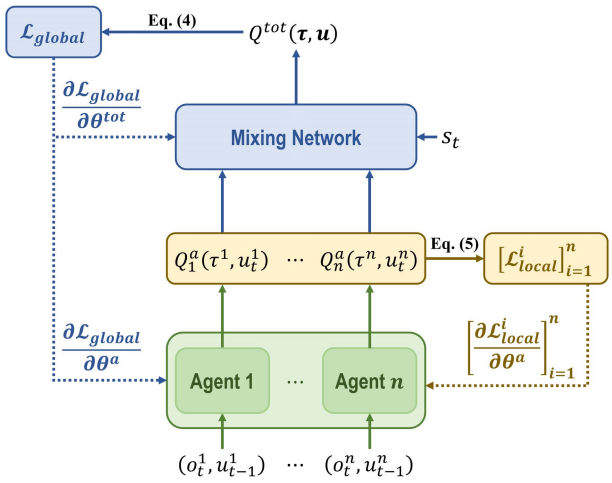


FIGURE 3. Overall network architecture of the QMIX with the proposed additional local loss scheme. Dotted lines indicate the direction in which the gradients calculated by (4) and (5) are backpropagated, respectively. Since the global loss $\mathcal{L}_{global}$ is backpropagated to the agent network through the mixing network, it does not significantly affect the agent network. However, the local loss $\mathcal{L}_{local}^a$ is backpropagated only to the agent network, enabling relatively clear feedback on each agent's experience.

D. REWARD FUNCTION

The environment returns the global reward and individual rewards according to the actions chosen by the agents per time step. The individual rewards are determined as follows:

$$r_t^i = \frac{1}{n}\left(d_{t-1}^i - d_t^i\right), \tag{2}$$

where d_t^i is the remaining Manhattan distance from agent i to its target position at time step t and n is the number of agents. In order for each agent to receive a good reward for each time step, they must choose an action towards their target position. On the other hand, the global reward is determined as follows:

$$r_t = \sum_{i=1}^{n} r_t^i. \tag{3}$$

The global reward allows a comprehensive evaluation of all the actions, but in some cases it is difficult to detect erroneous or sacrificial actions of a small number of agents. This reward design is advantageous in terms of improving the generalization performance of the agents.

IV. PROPOSED SCHEME

A. SEQUENTIAL ACTION MASKING

AGVs suffer from a huge disadvantage in that they can hit walls or collide with one another. The experiences created by this shortcoming are not useful and can even impede training. Therefore, we introduce action masking [25], [26], [27] to ensure that only high-quality experiences are stored in the replay buffer. In conventional action masking, the environment detects invalid actions of each agent and simultaneously notifies each agent of these at every time step. However, invalid actions may still be selected in the AGV warehouse.

Algorithm 1 Algorithm with Proposed Schemes

Initialize a agent network Q^a and a mixing network Q^{tot} with random parameters θ^a and θ^{tot}
Initialize target networks with parameters $\bar{\theta}^a \leftarrow \theta^a$ and $\bar{\theta}^{tot} \leftarrow \theta^{tot}$
Initialize a replay buffer $\mathcal{D}$
for episode $= 1$ to M **do**
  Initialize joint trajectory $\boldsymbol{\tau}$ and observe state s_t
  for $t = 1$ to T **do**
    for $i = 1$ to n **do**
      Receive agent i's observation o_t^i and available actions from the environment
      With probability ϵ:
        Select a random action u_t^i among the available actions
      Otherwise:
        Select $u_t^i \leftarrow \text{argmax}_u Q^a(o^i, u; \theta^a)$ among the available actions
    end
    Execute joint action $\boldsymbol{u}_t \equiv [u_t^i]_{i=1}^n$
    Get global reward r_t, individual rewards $[r_t^i]_{i=1}^n$, next state s_{t+1}, and joint next observation $\boldsymbol{o}_{t+1} \equiv [o_{t+1}^i]_{i=1}^n$
    Store transition $(s_t, \boldsymbol{o}_t, \boldsymbol{u}_t, r_t, [r_t^i]_{i=1}^n, s_{t+1}, \boldsymbol{o}_{t+1})$ in joint trajectory $\boldsymbol{\tau} \equiv [\tau^i]_{i=1}^n$
    Set $s_t \leftarrow s_{t+1}$
  end
  Store joint trajectory $\boldsymbol{\tau}$ in $\mathcal{D}$
  Sample a mini-batch $\mathcal{B}$ of S joint trajectories consisting of transitions $(s_j, \boldsymbol{o}_j, \boldsymbol{u}_j, r_j, [r_j^i]_{i=1}^n, s_{j+1}, \boldsymbol{o}_{j+1})$ from $\mathcal{D}$
  Calculate the global loss $\mathcal{L}_{global}$ **// described in Eq. (4)**
  for $i = 1$ to n **do**
    Make a mini-batch $\mathcal{B}^i$ of agent i's trajectory consisting of transitions $(o_j^i, u_j^i, r_j^i, o_{j+1}^i)$ from $\mathcal{B}$
    Calculate the local loss $\mathcal{L}_{local}^i$ **// described in Eq. (5)**
  end
  Update the parameters θ^a and θ^{tot} by minimizing the final loss $\mathcal{L}$ **// described in Eq. (6)**
  Every C episodes, copy parameters $\bar{\theta}^a \leftarrow \theta^a$ and $\bar{\theta}^{tot} \leftarrow \theta^{tot}$
end

Even if all the AGVs select a valid action, the valid action of each AGV may be changed to an invalid action because of the actions of other AGVs. To solve this problem, we propose a sequential action masking scheme instead of simultaneously applying action masking for each agent. Fig. 2 describes the sequential action masking process. At each time step, the environment notifies each agent in turn of its available actions. This process thoroughly ensures that agents do not collide with other agents as well as with walls or shelves. The sequential action masking scheme, which can be applied to any multi-agent RL algorithm such as IQL and QMIX, not only prevents all the possible collision cases of AGVs but also improves the performance of the algorithm by storing only efficient experiences in the replay buffer.

B. ADDITIONAL LOCAL LOSS

We propose a scheme that can be applied to QMIX while maintaining the CTDE framework. The final loss of the QMIX is calculated as the squared error between joint action value Q^{tot} and its target value for mini-batches as follows:

$$\mathcal{L}_{global} = \sum\nolimits_{\mathcal{B}} \left[\left(y^{tot} - Q^{tot}\left(\boldsymbol{\tau}, \boldsymbol{u}, s; \theta^{tot}\right)\right)^2 \right], \qquad (4)$$

where $y^{tot} = \mathrm{r} + \gamma \max_{\tilde{\boldsymbol{u}}} Q^{tot}\left(\boldsymbol{\tau}', \tilde{\boldsymbol{u}}, s'; \bar{\theta}^{tot}\right)$, $\boldsymbol{\tau}'$ is the next joint trajectory, s' is the next state, and $\bar{\theta}^{tot}$ is the parameters of the target network of the mixing network. The mixing network estimates the action value Q^a of each agent as the joint action value Q^{tot}, and the agent network estimates the action value Q^a of the available actions for each agent. The agent network interacts directly by receiving observations from the environment, whereas the mixing network relies heavily on the agent network. An incorrect estimation of the agent network leads to an incorrect estimation of the mixing network.

In the existing QMIX, the calculated global loss $\mathcal{L}_{global}$ as in (4) is backpropagated to the mixing network and the agent network. This backpropagation should be able to have a sophisticated impact on the mixing network and agent network. The mixing network directly participates in calculating the global loss $\mathcal{L}_{global}$ and is immediately backpropagated. However, the agent network simply provides its own estimations to the mixing network that computes the global loss $\mathcal{L}_{global}$. Even the backpropagation reaching the agent network is faint because it passes through the mixing network first. These two networks are strongly dependent on each other's estimations. To mitigate this dependency, we propose an

TABLE 1. List of hyper-parameters.

Hyper-parameters	Values
Optimizer	RMSProp
Discount factor γ	0.99
Learning rate	0.0005
Initial epsilon ϵ	1.0
Minimum epsilon ϵ	0.1
Epsilon ϵ annealing steps	5000
Episode steps T	300
Mini-batch episode size S	32
Replay buffer $\mathcal{D}$ capacity	500
Target network copy frequency C	10

additional local loss for the agent network formulated as:

$$\mathcal{L}^i_{local} = \sum\nolimits_{\mathcal{B}^i} \left[\left(y^i - Q^a \left(\tau^i, u^i; \theta^a \right) \right)^2 \right], \tag{5}$$

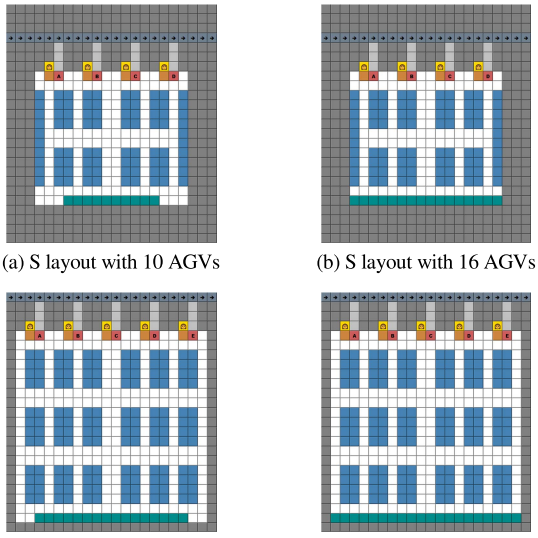
where $y^i = r^i + \gamma \max_{\bar{\boldsymbol{u}}^i} Q^a \left(\tau'^{,i}, \bar{u}^i; \bar{\theta}^a \right)$, $\tau'^{,i}$ is the next trajectory, and $\bar{\theta}^a$ is the parameters of the target network of the agent network. The local loss $\mathcal{L}^i_{local}$ is calculated as the squared error by using only the action value Q^a of the agent network for each agent. Although the agent network is shared, the observation-action history τ^i of each agent is maintained separately through the GRU [28], which is a component of the agent network. The final loss of the QMIX with the proposed additional local loss is defined as follows:

$$\mathcal{L} = \mathcal{L}_{global} + \sum\nolimits_{i=1}^{n} \mathcal{L}^i_{local}. \tag{6}$$

Fig. 3 shows the overall network architecture of the QMIX with the proposed additional local loss scheme along with the direction in which the global loss $\mathcal{L}_{global}$ and local loss $\mathcal{L}^i_{local}$ are backpropagated respectively.

The mixing network is the core idea of the CTDE, literally responsible for centralized training. However, it is harsh to establish the optimal policy of the agent network by the mixing network alone, without directly interacting with the environment. One of the most important updates for the agent network is specific feedback about their experiences. This feedback is not appropriate for the fact that the mixing network cannot know under what circumstances each agent estimated the action value Q^a. Furthermore, the QMIX solves the collaborative task of the entire team by combining the goals of individual agents through monotonic constraints. In this estimation, some experiences of agents may be ignored or lead to inaccurate feedback, as the main goal is to achieve maximum rewards in terms of the entire team.

The final loss consists of the global loss $\mathcal{L}_{global}$ and local loss $\mathcal{L}^i_{local}$ as in (6), and these two losses are backpropagated to the agent network. However, we expected that the influence of the local loss $\mathcal{L}^i_{local}$ is relatively much greater than that of the global loss $\mathcal{L}_{global}$. Therefore, one reasonable doubt can be raised that there will be no significant difference from the IQL which uses only the local loss $\mathcal{L}_{local}$ as the final loss.



(a) S layout with 10 AGVs (b) S layout with 16 AGVs

(c) L layout with 16 AGVs (d) L layout with 20 AGVs

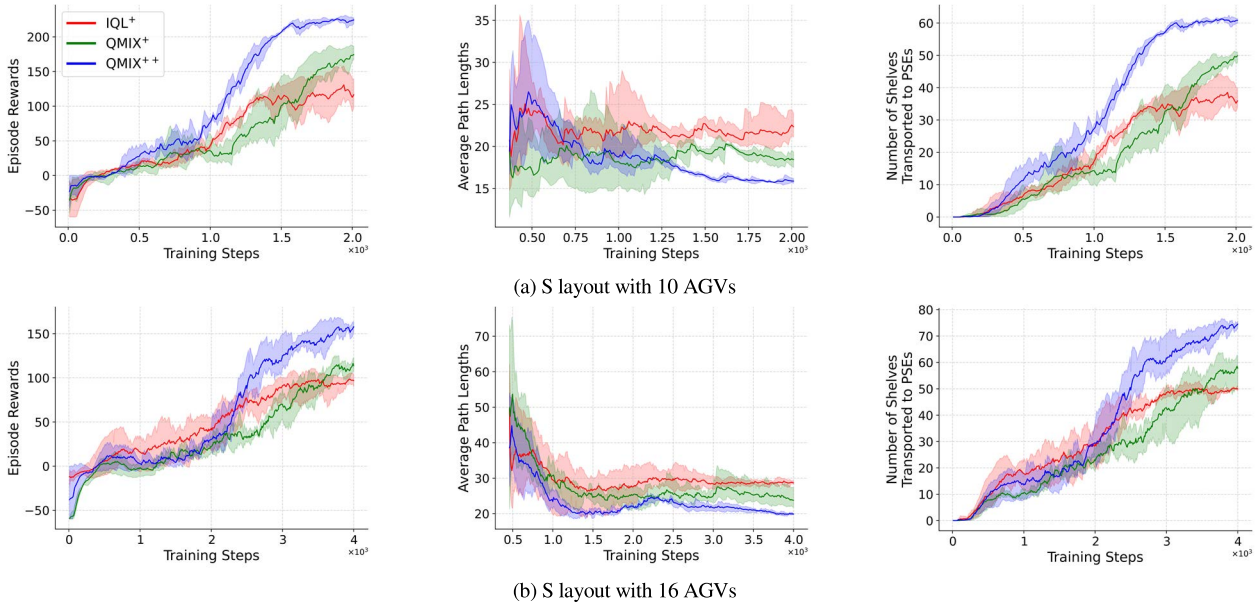
FIGURE 4. Layouts used to evaluate coordination performance. The S layout consists of 4 Picking Stations and 84 Shelves, and the L layout consists of 5 Picking Stations and 144 Shelves.

In Section V, to verify this we compare the proposed scheme based on the QMIX with the existing IQL.

We describe the whole algorithm with the proposed schemes in detail in Algorithm 1. By introducing sequential action masking on lines 7-13, all the collision cases are eliminated, but a new for statement is inevitable. Algorithm 1, a general RL algorithm, is for the training phase. However, in the inference phase, only lines 12 and 14 with for statements are required. In other words, decentralized decision-making is guaranteed because global information such as state is not required in the inference phase.

V. SIMULATIONS

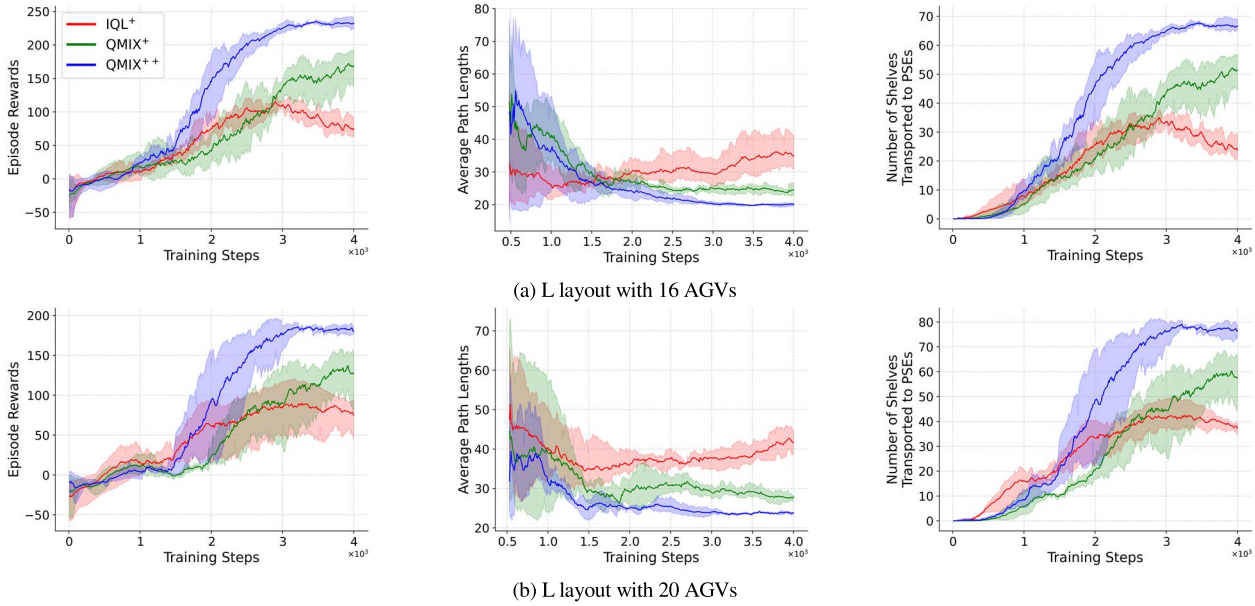
In this section, we performed two main simulations that demonstrate the superiority of the proposed schemes, which involve analysis of coordination performance and analysis of generalization performance. Since the IQL can be regarded as the most basic value-based multi-agent RL algorithm that is not equipped with cooperative methods, we set its performance as the baseline for measuring the effectiveness of the proposed schemes. Additionally, we draw on the performance results of the existing QMIX because the proposed scheme is based on it. As expected, they could not escape the curse of collisions between the AGVs when the sequential action masking was not applied. Therefore, only the sequential action masking without the additional local loss was applied to the existing QMIX and IQL. We emphasize once again that the sequential action masking can solve the problem of potential collisions in AGV warehouses, and that the additional local loss can significantly increase the performance of the entire system through clear feedback to individual agents under the assurance that no conflicts will occur. IQL$^+$ and QMIX$^+$ denote the existing IQL and QMIX to which the sequential action masking is applied, respectively, and



(a) S layout with 10 AGVs

(b) S layout with 16 AGVs

FIGURE 5. Training results on S layout.



(a) L layout with 16 AGVs

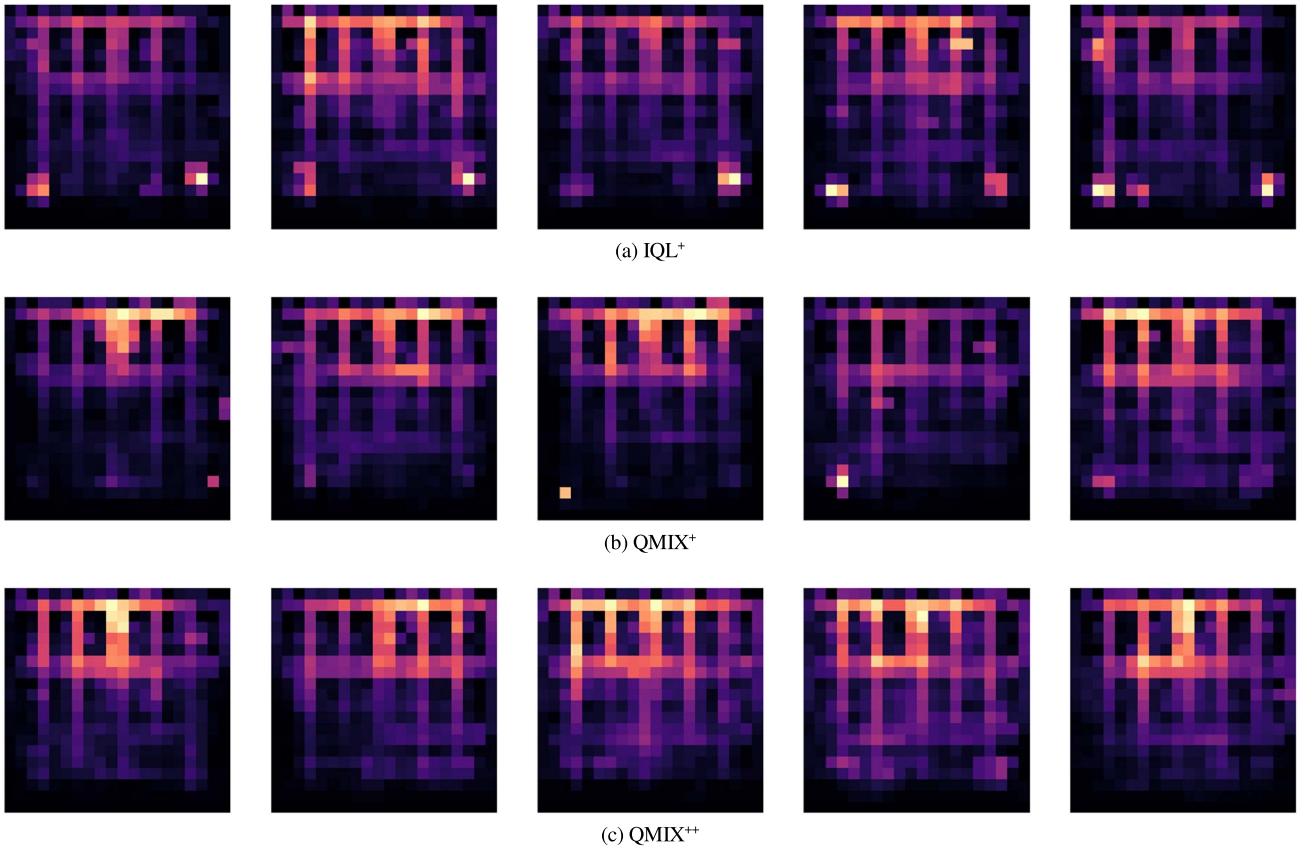
(b) L layout with 20 AGVs

FIGURE 6. Training results on L layout.

QMIX^{++} indicates that both the sequential action masking and the additional local loss are applied.

The hyper-parameters used in the simulations are presented in Table 1, and the fulfillment center layouts used in the simulations are illustrated in Fig. 4 to help in conveniently interpreting the results. All the fulfillment center layouts are 20 $\times$ 20 grid sizes, including no-access zones for the generalization performance simulation. Note that the state representation proposed in this paper is not limited to a specific size; any grid size can be used.

For a detailed evaluation, we adopted three metrics that are measured per episode: 1) episode rewards, which is the sum of the global rewards received; 2) average path lengths, which is the average of the time steps taken to complete each transport (there are three AGV states defined in Section III); and 3) number of shelves arrived at PSEs, which is the number of times AGVs transported shelves from the shelf storages to the picking stations. When 300 time steps were reached, the episode would end and a new one would begin. During training, for every 10 training steps, each metric is measured as the average value of the five separate test episodes in which all the agents take a greedy action. We plot the average of the five runs with 95% confidence intervals for each metric. Note that one training step means a single model update by the final



(a) IQL^{+}

(b) QMIX^{+}

(c) QMIX^{++}

FIGURE 7. Heat maps on L layout with 20 AGVs. Each heat map displays the path usage of AGVs for one episode, where locations with high path usage are relatively bright. Note that AGVs without shelf can pass under the shelves in the shelf storage.

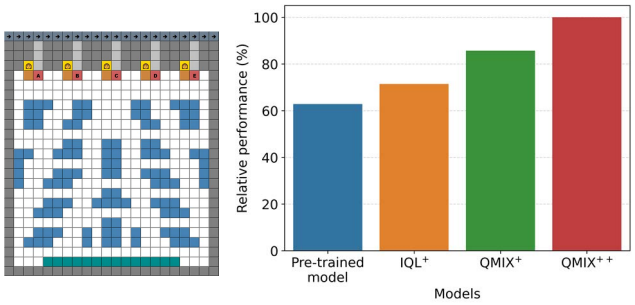
loss that aggregates the global loss and local losses as in (6), and it is calculated using the experience sampled from the replay buffer.

A. ANALYSIS OF COORDINATION PERFORMANCE

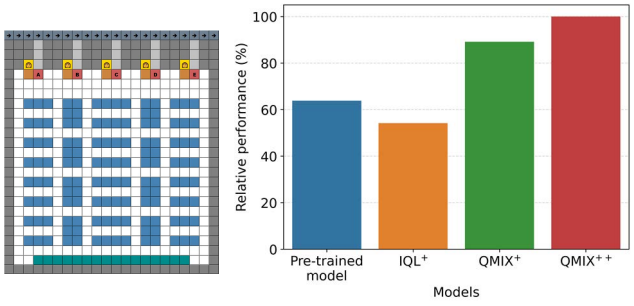
In this simulation, we show the effect of communication between agents on their coordination performance as seen in Fig. 5 and 6. Both figures plot the results in the same way except for the fulfillment center layout to further analyze the changes in coordination performance according to the scale of the fulfillment center layout. We also plot the results of reducing the number of AGVs for the S and L layouts. Even if all the other conditions were equal, the adjustment of the number of AGVs changed their required cooperative capability. As described in Section IV, we eliminated all the collision cases by introducing sequential action masking. However, the AGVs still cannot pass through each other. To make matters worse, the space of the fulfillment center layout is limited, and the dynamic path control of AGVs is required.

As presented in Fig. 4, we deliberately created spaces between shelves or between shelves and no-access zones as a one-way path. Ultimately, to accomplish the task efficiently, it is necessary to communicate which AGVs will use a one-way path for a specific period. If all the AGVs pursued their own interests only, they would never be able to efficiently solve this core challenge. Although some AGVs suffered losses by making concessions or using suboptimal options, it was possible to increase the final performance of the entire system through these sacrifices. This collaboration is a hidden challenge that AGVs must overcome and explains the need to apply real-time path control.

The reason IQL^{+}'s training is very unstable is that the AGVs see each other only as obstacles because of the absence of cooperative methods between them. Without awareness of each other's existence, bottlenecks occur constantly in one-way paths, and the AGVs frequently interfere with each other's paths. The training of QMIX^{+} shows that although the AGVs tried to recognize each other and collaborate, they were still unstable and had poor performance. This represents a chronic problem that sophisticated control of individual agents is difficult in the existing QMIX, which is obsessed only with the global reward for the entire team. In contrast, in QMIX^{++}, the AGVs are finely controlled based on clear individual feedback. This enables smooth communication between the AGVs and leads to stable and superior performance. Note that the performance difference between QMIX^{+} and QMIX^{++}, which is the performance improvement obtained by alleviating an inherent problem of the existing QMIX with proposed additional local loss scheme, is comparable to or greater than the performance difference between IQL^{+} and QMIX^{+}.



(a) Results on the first unseen layout



(b) Results on the second unseen layout

FIGURE 8. Generalization performance for unseen fulfillment center layouts. The pre-trained model is the one that is trained on the S layout and then on the L layout using QMIX^{++}. The performance of the pre-trained model shown on the leftmost bars in the bar graphs is respectively measured by inference without additional training on each of the two unseen layouts. The remaining bars represent the performance of training directly on the two unseen layouts with each scheme, namely IQL^{+}, QMIX^{+}, and QMIX^{++}, where the performance of QMIX^{++} is set to 100% in terms of number of shelves arrived at PSEs metric.

At first glance, the S and L layouts are similar in structure with only the size being different as shown in Fig. 4. We intended that the configurations of the shelf storages, picking stations, and AGVs would be similar. However, the complexity of these two fulfillment center layouts differs significantly because of the size of the fulfillment center layout. As expected, QMIX^{++} shows more stability and better performance than those of IQL^{+} and QMIX^{+} in terms of adaptability according to the scale of the layout.

Fig. 7 shows the path usage distribution of the AGVs as heat maps of how evenly they move along their transport path in the L layout. Since the picking stations are located north of the layout, it is expected that most of the transport paths of the AGVs will be north of the layout. In the heat maps of IQL^{+}, the north of the layout is relatively less bright owing to the poor cooperation between the AGVs. In addition, several AGVs that are idle in a corner are captured as a result of erroneous training owing to the lack of communication between them. Since QMIX^{+} supports communication between the AGVs, a smooth path occupancy is achieved, and the north of the layout is generally bright. However, there are still a handful of AGVs striking the corner under QMIX^{+}, although less than that under IQL^{+}. In contrast, the heat maps of QMIX^{++} show a more flexible transport path usage for AGVs. In particular, idle AGVs, which are generated frequently in IQL^{+} or QMIX^{+}, are not brought out because of the clear feedback for each AGV.

B. ANALYSIS OF GENERALIZATION PERFORMANCE

In this study, we formulated AGV warehouse tasks such as those in the fulfillment center as a Dec-POMDP. This enables AGVs to make decentralized decisions using only their observations. Although the states, which represent the overall information of the fulfillment center layout, are used in the training phase for specific fulfillment center layouts, they are not used in the inference phase after the training phase. Thus, we justify that the proposed design supports robust generalization performance by inferring the transport path of AGVs in unseen layouts from a model trained on specific fulfillment center layouts. Fig. 8 shows that the generalization performance of the proposed QMIX^{++} for the unseen layouts is approximately 60% compared with that of QMIX^{++} trained on the new unseen layout. Even, results that exceed the performance of IQL^{+} are derived as shown in Fig. 8(b).

In the real world, when building a new fulfillment center or changing its layout, the proposed scheme has a disadvantage in that the control model of AGVs needs to be trained from scratch. Moreover, if the fulfillment center layout is extremely large or complex, the training time increases exponentially. However, if we face these problems, we can alleviate the struggles faced in new training by reusing models that have been trained on multiple fulfillment center layouts.

VI. DISCUSSION

In terms of implementation, the proposed additional local loss scheme can be applied to any value-based multi-agent RL algorithm, including QMIX, by simply adding the local loss to the final loss. Similarly, the proposed sequential action masking scheme can be also applied to any multi-agent RL algorithm with an intuitive process as shown in Fig. 2. In other words, the proposed schemes are easy to be implemented since their inherent features are orthogonal to the main algorithm.

When each AGV's policy model is trained, on the other hand, the complexity of the proposed schemes is usually high because the centralized mixing architecture is used according to the CTDE framework. After training phase, however, the trained policy model is capable of real-time control that requires very short periods of control signals, and thus the complexity of the proposed schemes is not high at the execution time, which is the main feature of most RL-based algorithms.

The proposed schemes are different from the existing RL-based ones because they did not significantly consider collaboration between AGVs when designing their RL algorithm. For example, efficient occupation of one-way paths can never be achieved without cooperation between AGVs, which

greatly affects productivity. We believe that this collaboration capability is important for most multi-AGV systems, so that the proposed solution is sufficiently practical for most fulfillment systems with multiple AGVs.

VII. CONCLUSION

In this study, we studied the cooperative movement control of multiple AGVs working together in a confined space to achieve a common goal. We formalized this task as a Dec-POMDP to solve it by using multi-agent RL. The proposed schemes enabled individually coordinated path control to improve the performance of the entire system. The results are presented with three metrics: 1) episode rewards, 2) average path lengths, and 3) number of shelves arrived at PSEs. We also provided heat maps to investigate transport path usage. The proposed additional local loss scheme significantly improved the performance of the algorithm through specific individual feedback for each agent. In particular, the proposed sequential action masking scheme eliminated all the possible collision cases. In the near future, we will apply the federated RL to this task, where multiple layouts are trained simultaneously across many agents, to further improve the generalization performance.

REFERENCES

[1] C. Wang and J. Mao, "Summary of AGV path planning," in *Proc. 3rd Int. Conf. Electron. Inf. Technol. Comput. Eng. (EITCE)*, Oct. 2019, pp. 332–335.

[2] M. Chen, T. Wang, K. Ota, M. Dong, M. Zhao, and A. Liu, "Intelligent resource allocation management for vehicles network: An A3C learning approach," *Comput. Commun.*, vol. 151, pp. 485–494, Feb. 2020.

[3] M. Chen, W. Liu, T. Wang, A. Liu, and Z. Zeng, "Edge intelligence computing for mobile augmented reality with deep reinforcement learning approach," *Comput. Netw.*, vol. 195, Aug. 2021, Art. no. 108186.

[4] L. Buşoniu, R. Babuška, and B. De Schutter, "Multi-agent reinforcement learning: An overview," in *Innovations in Multi-Agent Systems and Applications*, vol. 1. Berlin, Germany: Springer, 2010, pp. 183–221.

[5] X. Bai, A. Fielbaum, M. Kronmuller, L. Knoedler, and J. Alonso-Mora, "Group-based distributed auction algorithms for multi-robot task assignment," *IEEE Trans. Autom. Sci. Eng.*, early access, May 25, 2022, doi: 10.1109/TASE.2022.3175040.

[6] J. Hu, H. Niu, J. Carrasco, B. Lennox, and F. Arvin, "Fault-tolerant cooperative navigation of networked UAV swarms for forest fire monitoring," *Aerosp. Sci. Technol.*, vol. 123, Apr. 2022, Art. no. 107494.

[7] J. Cui, Y. Liu, and A. Nallanathan, "Multi-agent reinforcement learning-based resource allocation for UAV networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 2, pp. 729–743, Feb. 2020.

[8] X. Li, J. Zhang, J. Bian, Y. Tong, and T. Liu, "A cooperative multi-agent reinforcement learning framework for resource balancing in complex logistics network," in *Proc. 18th Int. Conf. Auton. Agents MultiAgent Syst.*, 2019, pp. 1–14.

[9] X. Li, X. Hu, W. Li, and H. Hu, "A multi-agent reinforcement learning routing protocol for underwater optical sensor networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–7.

[10] F. A. Oliehoek, M. T. J. Spaan, and N. Vlassis, "Optimal and approximate Q-value functions for decentralized POMDPs," *J. Artif. Intell. Res.*, vol. 32, no. 1, pp. 289–353, 2008.

[11] J. J. Enright and P. R. Wurman, "Optimization and coordinated autonomy in mobile fulfillment systems," in *Proc. AAAI Workshop Automated Action Planning Auto. Mobile Robots*, 2011, pp. 33–38.

[12] J. Bae and W. Chung, "A heuristic for a heterogeneous automated guided vehicle routing problem," *Int. J. Precis. Eng. Manuf.*, vol. 18, no. 6, pp. 795–801, Jun. 2017.

[13] Z. Han, D. Wang, F. Liu, and Z. Zhao, "Multi-AGV path planning with double-path constraints by using an improved genetic algorithm," *PLoS ONE*, vol. 12, no. 7, Jul. 2017, Art. no. e0181747.

[14] Y. Lian and W. Xie, "Improved a multi-AGV path planning algorithm based on grid-shaped network," in *Proc. Chin. Control Conf. (CCC)*, Jul. 2019, pp. 2088–2092.

[15] R. Kamoshida and Y. Kazama, "Acquisition of automated guided vehicle route planning policy using deep reinforcement learning," in *Proc. 6th IEEE Int. Conf. Adv. Logistics Transp. (ICALT)*, Jul. 2017, pp. 1–6.

[16] Y. Yang, L. Juntao, and P. Lingling, "Multi-robot path planning based on a deep reinforcement learning DQN algorithm," *CAAI Trans. Intell. Technol.*, vol. 5, no. 3, pp. 177–183, 2020.

[17] L. C. Garaffa, M. Basso, A. A. Konzen, and E. P. de Freitas, "Reinforcement learning for mobile robotics exploration: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 12, 2021, doi: 10.1109/TNNLS.2021.3124466.

[18] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.

[19] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[20] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proc. 10th Int. Conf. Mach. Learn.*, 1993, pp. 330–337.

[21] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. F. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. 17th Int. Conf. Auton. Agents Multiagent Syst.*, Stockholm, Sweden, 2018, pp. 1–17.

[22] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn.*, Stockholm, Sweden, 2018, pp. 4295–4304.

[23] D. Ha, A. Dai, and Q. V. Le, "Hypernetworks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–29.

[24] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs* (SpringerBriefs in Intelligent Systems). Cham, Switzerland: Springer, 2016.

[25] O. Vinyals *et al.*, "StarCraft II: A new challenge for reinforcement learning," 2017, *arXiv:1708.04782*.

[26] D. Ye, Z. Liu, M. Sun, B. Shi, P. Zhao, H. Wu, H. Yu, S. Yang, X. Wu, Q. Guo, Q. Chen, Y. Yin, and H. Zhang, "Mastering complex control in MOBA games with deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 6672–6679.

[27] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," 2020, *arXiv:2006.14171*.

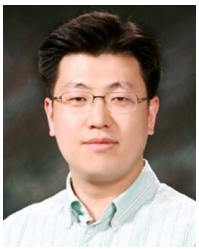
[28] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. NIPS Workshop Deep Learn.*, 2014, pp. 1–9.



HO-BIN CHOI received the B.S. and M.S. degrees in computer science engineering from the Department of Computer Science Engineering, Korea University of Technology and Education, Cheonan, South Korea, in 2019 and 2021, respectively, where he is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering. His current research interests include cooperative multi-agent systems, non-stationary environment analysis, game development, and reinforcement learning.



JU-BONG KIM received the B.S. and M.S. degrees in computer science engineering from the Department of Computer Science Engineering, Korea University of Technology and Education, Cheonan, South Korea, in 2017 and 2019, respectively, where he is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering. His current research interests include deep learning and reinforcement learning across various domains, especially device control and blockchain assets.



YOUN-HEE HAN (Member, IEEE) received the B.S. degree in mathematics and the M.S. and Ph.D. degrees in computer science and engineering from Korea University, Seoul, South Korea, in 1996, 1998, and 2002, respectively. From 2002 to 2006, he was a Senior Researcher with the Next Generation Network Group, Samsung Advanced Institute of Technology. He has been a Professor with the School of Computer Science and Engineering, Korea University of Technology and Education, Cheonan, South Korea, since 2006. His activities have been focusing on mobility management, media independent handover, and cross-layer optimization for efficient mobility support, since 2002. He has published approximately 300 research articles on the theory and application of mobile computing and has led 50 patents on information and communication technology domain. His past research interests include theory and application of computer networks, including protocol design and mathematical analysis, mobile sensor/actuator networks, social network analysis, machine learning, deep learning, and reinforcement learning. He is currently very interested in artificial intelligence technology, especially reinforcement learning. His research interests include reinforcement learning algorithms, and continues to work on optimizing the performance by applying reinforcement learning agents to various fields, such as intelligent networking on 5G and 6G, the IoT (Internet of Things), smart factory, and financial engineering. He has also made several contributions in IETF and IEEE standardization.



SE-WON OH received the B.S. and M.S. degrees from the Pohang University of Science and Technology (POSTECH), South Korea, in 1999 and 2001, respectively, and the Ph.D. degree in computer science and engineering from Chungnam National University (CNU), Daejeon, South Korea, in 2018. He has been working as a Principal Researcher for Intelligent Convergence Research Laboratory in Electronics and Telecommunications Research Institute (ETRI), Daejeon, South Korea, since 2001. He has been involved in several Research Projects on high-performance software systems such as RFID event management, sensor networking middleware, Internet of Things platform, and knowledge-fusion solutions that integrate legacy applications with various data resources and sensors. His research interest includes machine learning applications for anomaly detection and intelligent safety services.



KWIHOON KIM received the B.S., M.S., and Ph.D. degrees from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 1998, 2000, and 2019, respectively. He is currently a Professor with the Department of Artificial Intelligence Convergence Education, Korea National University of Education (KNUE), South Korea. He worked in LG DACOM 2000∼2005. From 2005 to 2020, he was a Principle Researcher with Electronics and Telecommunications Research Institute (ETRI). His research interests include AI convergence education, intelligent edge computing, reinforcement learning, and knowledge-converged intelligent service.